FINN for Real-Time Embedded Perception

Michał Daniłowicz, Bartłomiej Bańka, Konrad Lis, Mateusz Wąsala, Tomasz Kryjak

Motivation

We present a summary of our group's recent research using and extending the FINN framework for real-time embedded perception on SoC FPGAs. Effective perception of the environment is the foundation of autonomy in modern mobile robotics. The ability to accurately detect and track objects and estimate the position of the platform are key issues in many modern applications, such as navigation of autonomous robots and advanced driver assistance systems. First, we extend FINN to enable deployment of modern YOLOv8 detectors and perform design space exploration to balance detection quality and resource consumption. Second, we implemented the SuperPoint network in FINN - a convolutional neural network that jointly detects feature points and computes their descriptors in a single forward pass. Third, we introduce a sparse convolution engine designed specifically for real-time, LiDAR-based 3D object detection on embedded FPGAs. Fourth, we integrated Power-of-Two quantization into the FINN ecosystem by implementing a bitshift-accumulate variant of the MVU, replacing multiply-accumulate with shift-accumulate operations.

Contribution

Real-time object detection with YOLOv8

Efficient, low-power object detection is critical for real-time perception in autonomous mobile robotics. To deploy modern one-stage detectors on SoC FPGAs, we first extended the FINN framework by introducing a channel split operation, mixed-precision concatenation nodes, and custom streamlining operations tailored for YOLOv8 topologies. With these extensions, we conducted a design space exploration across varying input resolutions, model sizes, and quantization levels. The hardware parallelization of each configuration was normalized to a target processing speed of 30 fps to establish a standardized baseline for performance comparison. For this experiment, we implemented all key accelerator modules using LUTs, to reduce resource consumption to a single metric and simplify the analysis. The results are shown in Figure 1.

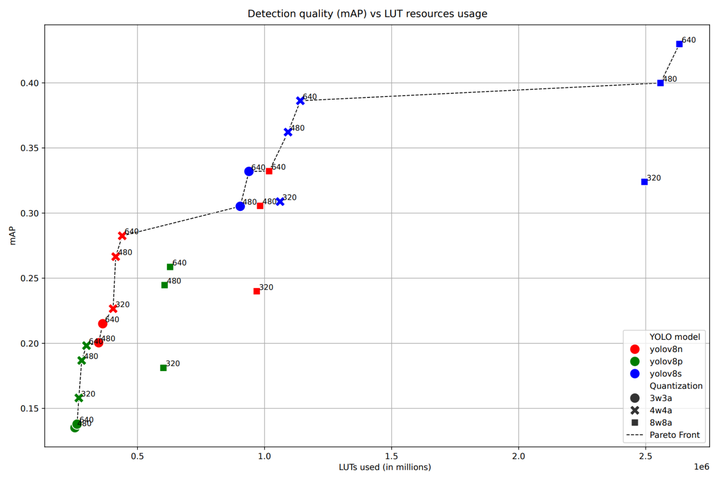
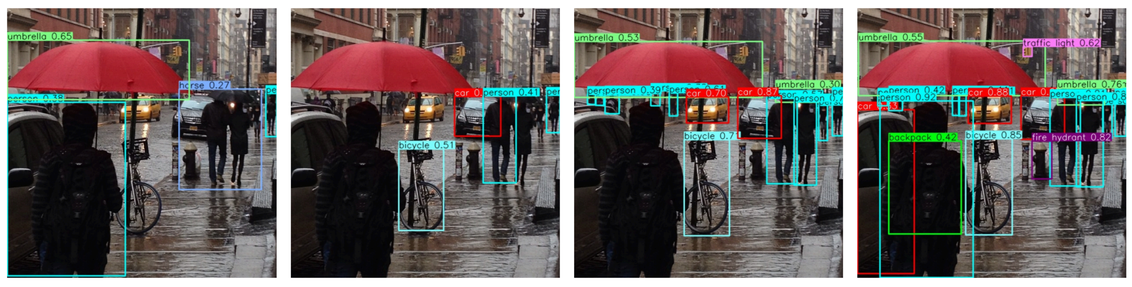


Figure 1. Pareto analysis of the YOLOv8 configurations: detection quality (COCO mAP) versus estimated logic resource usage (LUT). The Pareto front, indicating the set of optimal solutions, is marked with a dashed line. The numbers next to the data points represent the input image resolution.

We tested three variants of the YOLOv8 detector on the COCO val2017 dataset: pico (1,5M parameters), nano (3,2M parameters) and small (11,2M parameters). Our resulting Pareto front analysis demonstrates that prioritizing high-resolution inputs (640 pixels for the longer edge) consistently yields the best detection quality for a given hardware budget. Furthermore, the results indicate that it is beneficial to utilize a larger model architecture at the expense of numerical precision, with 4-bit quantization emerging as the sweet spot between accuracy and logic resource utilization. For comparison, we visualize selected configurations in Figure 2. The proposed extensions to FINN were merged into the development branch and a pre-built YOLOv8n demo for Alveo u55c is available for community use [1].



(a) 3w3a_yolov8p_480, 0.135 mAP (b) 4w4a_yolov8n_320, 0.227 mAP (c) 4w4a_yolov8n_640, 0.283 mAP (d) 8w8a_yolov8s_640, 0.430 mAP

Figure 2. Visual comparison of obect detection quality for four sample detector configurations on a test image. The specific model configuration and its corresponding mean Average Precision (mAP) on the COCO val2017 dataset are indicated below each image.

Visual Odometry with Superpoint network

Visual odometry is a technique that estimates a robot's trajectory and position by calculating the displacement between successive images of its operating environment. A fundamental component of this process is feature point matching, which associates distinct visual landmarks across consecutive frames to track movement - visualized on Figure 3. Because feature point detection and description represent the most computationally demanding stage of the visual odometry pipeline, accelerating this step is critical.



Figure 3. By extracting and matching stable feature points between successive frames, the system can calculate spatial displacement and determine the vehicle's trajectory, such as the right turn shown here.

To address this, we focused on SuperPoint, a widely adopted convolutional, self-supervising neural network that simultaneously generates feature points and their corresponding descriptors. By leveraging the flexibility of the FINN framework, we successfully implemented a 3-bit quantized model. This hardware deployment required extending FINN to support external model parameters for convolutional layers (previously available only for fully-connected layers). This extension allows our model's parameters to be stored in the processor system's RAM and streamed to the programmable logic accelerator via DMA. Ultimately, we deployed this design using two different parallelization levels, achieving real-time processing throughputs of 27 fps on the Kria KV260 platform and 54 fps on the ZCU102 development board. Our implementation using FINN outperforms state-of-the-art solutions in the field [2].

Sparse convolutions for LiDAR-based 3D object detection

LiDAR-based 3D object detection is a critical perception task for autonomous navigation, providing highly accurate spatial representations of a vehicle's surroundings. As visualized in Figure 4, this sensory data is inherently sparse. Because the vast majority of the 3D grid contains empty space, applying standard dense convolutions wastes significant computational resources and memory bandwidth on zero-value regions.

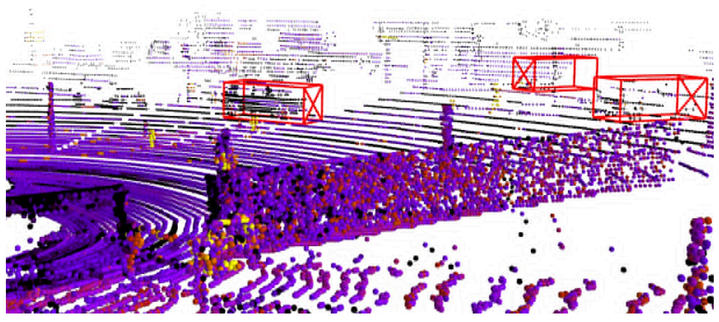


Figure 4. 3D vehicle detection on sparse LiDAR data. The inherently sparse nature of the point cloud highlights the need for specialized sparse convolution architectures.

To address this inefficiency, we are currently working on SCARL: the first open-source FPGA implementation of a 2D sparse convolution accelerator designed to operate directly on a sparse input format. SCARL processes data in COO format - stream of meaningful features and correspoinding pixel coordinates. For the core matrix computations, the design utilizes efficient MVU kernels sourced from finn-rtllib. Furthermore, because our target model relies on 8-bit quantization, we developed a custom activation requantization module that directly multiplies activations with quantization scales. Nevertheless, SCARL's internal dataflow remains entirely compatible with FINN's native Thresholding modules, allowing them to be swapped in directly for requantization in lower-precision networks. The dynamic parametrization of the accelerator via an AXI4-Lite interface enables the accelerator to operate in either iterative or full dataflow modes. Running at 200 MHz, SCARL achieves a computational throughput of 330 GMAC/s, successfully enabling real-time 3D object detection on low-power embedded platforms.

Power-of-Two quantization

Power-of-Two (PoT) quantization presents a compelling alternative to standard uniform quantization. By concentrating higher numerical resolution closer to zero, PoT better aligns with the typical bell-shaped distribution of neural network parameters. Crucially, this approach has the potential to be highly hardware-efficient, as it replaces resource-intensive multiplication with simple bitshift operations during network inference. Furthermore, at extremely low bit-widths (e.g., 4-bit precision), PoT quantization can yield better performance and higher accuracy than uniform quantization for tasks such as image classification. To leverage these benefits, we integrated PoT support into the FINN ecosystem by implementing a custom bitshift-accumulate variant of FINN's MVU. Initial experiments demonstrate significant logic savings. Our bitshift MVU requires roughly half the LUT utilization compared to the standard finn-hlslib LUT-based MVU. Moreover, it achieves a LUT footprint comparable to the finn-rtllib DSP-based MVU, but does so while consuming no DSP resources. The post-synthesis resource consumption comparing these implementations is summarized in Table 1.

Table 1. Post-Synthesis resource utilization of 4-bit MVAU architectures (equal parallelization level, PE = SIMD = 32)

	LUT	FF	DSP
MVU_rtl	14498	17904	256
MVU_hls	32426	13492	0
MVU_shift	15656	19597	0

Discussion & Future Work

In summary, our research demonstrates that FINN is an exceptionally versatile framework for embedded, AI-based perception. Beyond enabling rapid design space exploration, its modular architecture proved to be highly customizable, allowing us to extend the framework with novel, specialized hardware modules tailored to our specific scientific needs. By adapting FINN to handle everything from mixed-precision YOLO detectors to sparse LiDAR point clouds, we highlight its broad applicability for next-generation robotic perception. To build on these results, our to-do list includes:

- **SCARL Compiler Integration:** Integrate SCARL's iterative and dataflow architectures directly into the FINN compiler to enable fast prototyping and rapid deployment of new sparse convolutional networks.
- **End-to-End PoT Deployment:** Get a working PoT example on hardware. We plan to deploy a PoT-quantized YOLOv8 detector to establish a direct comparison with our uniform-quantized baseline.
- **QONNX Extensions:** Introduce new PoT-specific datatypes and nodes into the QONNX intermediate representation to fully support the PoT flow. Additionally, we need to figure out how to extend the compiler's dataflow paradigm so it can natively grasp and compile the COO format for sparse data.

References

[1] https://github.com/Xilinx/finn-examples/pull/113
[2] Wasala M., et al. "Hardware-Aware Feature Extraction Quantisation for Real-Time Visual Odometry on FPGA Platforms"
[3] Przewlocka-Rus D., et al. "Power-of-Two Quantization for Low Bitwidth and Hardware Compliant Neural Networks"

Contact: Michał Daniłowicz, danilowi@agh.edu.pl

. The research was supported by programme "Excellence initiative – research university" for the AGH University of Science and Technology.
. We gratefully acknowledge Polish high-performance computing infrastructure PLGrid (HPC Center: ACK Cyfronet AGH) for providing computer facilities and support within computational grant no. PLG/2025/018652